



INSTITUTO TECNOLÓGICO Y DE ESTUDIOS

SUPERIORES DE MONTERREY

Actividad 4.2 Ejercicio de programación 1

**Adriana González Ugalde
A00194173**

TC4017 Pruebas de software y aseguramiento de la calidad

Dr. Gerardo Padilla Zárate

8 de febrero de 2026

Tabla de contenido

OBJETIVO Y AMBIENTE DE DESARROLLO	1
ESTRUCTURA DEL REPOSITORIO.....	1
PROGRAMA 1 – COMPUTE STATISTICS.....	1
Descripción del problema:	2
Requerimientos recibidos	2
Consideraciones y especificaciones adicionales sobre la implementación	2
Ejecución del programa	3
Resultados obtenidos	3
Verificación con pylint	5
PROGRAMA 2 – CONVERTER.....	5
Descripción del problema:	5
Requerimientos recibidos	5
Consideraciones y especificaciones adicionales sobre la implementación	6
Ejecución del programa	8
Resultados obtenidos	8
Verificación con pylint	9
PROGRAMA 3 – COUNT WORDS	9
Descripción del problema:	9
Requerimientos recibidos	9
Consideraciones y especificaciones adicionales sobre la implementación	10
Ejecución del programa	11
Resultados obtenidos	11
Verificación con pylint	12
CONCLUSIONES / REFLEXIONES.....	13
REFERENCIAS (FORMATO APA)	14

Objetivo y ambiente de desarrollo

El objetivo de este documento es presentar la implementación, ejecución y validación de tres ejercicios de programación desarrollados en Python, aplicando criterios de calidad de software y utilizando la herramienta pylint para verificar el cumplimiento del estándar PEP-8.

Los programas fueron desarrollados utilizando Python y ejecutados desde la línea de comandos mediante la terminal integrada de Visual Studio Code.

Para la verificación de calidad de código se utilizó la herramienta pylint.

Estructura del Repositorio

El **repositorio** se encuentra en Git, con el nombre **A00194173-Pruebas-y-Calidad** y tiene la estructura necesaria para contener este ejercicio y los posteriores de la materia, siguiendo la sugerencia del profesor Gerardo Padilla, lo cual me parece muy bueno para que sea más uniforme para encontrar las evidencias del ejercicio.

Este ejercicio en particular, *4.2. Ejercicio de programación 1*, se encuentra en la **carpeta A00194173_A4.2**, nombre solicitado en el ejercicio.

La estructura de directorios del repositorio para esta actividad es:

A00194173-Pruebas-y-Calidad	← Repositorio
└─ A00194173_A4.2	← Actividad 4.2
├─ P1	← Programa 1
├─ Source	← Código fuente
├─ Test	← Datos de prueba
└─ Results	← Resultados pbas: .txt y screenshots
├─ P2	← Programa 2
├─ Source	← Código fuente
├─ Test	← Datos de prueba
└─ Results	← Resultados pbas: .txt y screenshots
└─ P3	← Programa 3
├─ Source	← Código fuente
├─ Test	← Datos de prueba
└─ Results	← Resultados pbas: .txt y screenshots

Programa 1 – Compute Statistics

Descripción del problema:

El ejercicio consiste en desarrollar un programa en Python que lea un archivo de texto con valores numéricos y calcule estadísticas descriptivas básicas, manejando (excluyendo) datos inválidos y reportando los resultados tanto en pantalla como en archivos de salida.

Requerimientos recibidos

Los requerimientos implementados son los detallados en el documento de la asignación y que especifican:

Req1. The program shall be invoked from a command line. The program shall receive a file as parameter. The file will contain a list of items (presumable numbers).

Req 2. The program shall compute all descriptive statistics from a file containing numbers. The results shall be print on a screen and on a file named StatisticsResults.txt. All computation MUST be calculated using the basic algorithms, not functions or libraries.

The descriptive statistics are mean, median, mode, standard deviation, and variance.

Req 3. The program shall include the mechanism to handle invalid data in the file. Errors should be displayed in the console and the execution must continue.

Req 4. The name of the program shall be
computeStatistics.py

Req 5. The minimum format to invoke the program shall be as follows:
python computeStatistics.py fileWithData.txt

Req 6. The program shall manage files having from hundreds of items to thousands of items.

Req 7. The program should include at the end of the execution the time elapsed for the execution and calculus of the data. This number shall be included in the results file and on the screen.

Req 8. Be compliant with PEP8.

Consideraciones y especificaciones adicionales sobre la implementación

Adicional a los requerimientos de la asignación, se recibieron precisiones, y se tomaron decisiones sobre la implementación, para poder cumplir con los requerimientos y con las salidas esperadas.

A continuación se detalla cuáles fueron.

1. Medidas estadísticas utilizadas

Se indicó que las medidas estadísticas utilizadas, cumpliendo con las indicaciones, son: **media, mediana, moda, varianza poblacional y desviación estándar poblacional.**

2. Precisiones sobre la moda

La moda se calculó considerando la frecuencia de aparición de los valores.

Si uno o más valores presentan la frecuencia máxima, la población tendría una o más modas. En este caso, **se reportó solamente la primera moda encontrada, para coincidir con los datos de salida esperados en los casos en los que había más de una moda.**

Cuando todos los valores aparecen una sola vez, se concluye correctamente que no existe moda, y se reporta como NA.

3. Varianza y desviación estándar

La varianza y la desviación estándar se calcularon utilizando la definición de **varianza poblacional como lo indicó el profesor**, dividiendo entre N (no entre N-1 que sería para la varianza de la muestra),

Se observó **que los valores de salida esperados corresponden a la varianza muestral (N-1), lo cual representa una inconsistencia con el requerimiento original.** En este ejercicio se mantuvo el cálculo de **varianza poblacional conforme a la especificación.**

4. Nombre del archivo de código fuente

El requerimiento solicita que el archivo de código fuente tenga el nombre **computeStatistics.py**, cuya estructura **no es adecuada de acuerdo** a los estándares de PEP8, por lo que **se cambió a notación snake case: compute_statistics.py**

Ejecución del programa

El programa se ejecutó desde la línea de comandos en la consola, como fue indicado, recibiendo como parámetro el archivo de prueba correspondiente (TC1.txt, TC2.txt... TC7.txt).

Ejemplo:

```
python compute_statistics.py ..\Test\TC1.txt
```

Resultados obtenidos

El nombre del archivo de salida es el solicitado en los requerimientos, **StatisticsResults**, y para facilitar la trazabilidad de los casos de prueba, **el nombre del**

archivo de salida incluye el identificador del archivo de entrada, generando un archivo de resultados independiente por cada caso de prueba.

Ejemplo:

Entrada: TC1.txt

Salida: **StatisticsResultsTC1.txt**

En el **subfolder Screenshots del folder Results** se encuentran las **capturas de pantalla de los resultados de las pruebas, con nombres que incluyen Ejec** para indicar ejecución, del tipo 2 Ejec..., 4 Ejec... m 7 Ejec.... **Los numerales al inicio del nombre de archivo indican orden en la ejecución de los diferentes casos de prueba y de Pylint.**

Los archivos de resultados y screenshots se encuentra en los siguientes directorios, y el numeral al inicio indica orden de ejecución:

.	
.	
.	
P1	← Programa 1
└─ Source	← Código fuente
└─ Test	← Código fuente
└─ Results	← Resultados pbas: .txt y screenshots
└─ StatisticsResultsTC1.txt	← Resultados del TC1
...	
└─ StatisticsResultsTC7.txt	← Resultados del TC7
└─ Screenshots	← Capturas de pantalla
└─ 1 Pylint P1-1.png	← Ejecución de Pylint
...	
└─ 1 Pylint P1-4.png	← Ejecución de Pylint
└─ 2 Ejec. TC1.png	← Ejecución de Código
└─ 2 Ejec. TC1-2.png	← Ejecución de Código
└─ 3 Pylint P1-5.png	← Ejecución de Pylint
└─ 4 Ejec. TC1-3 (desp P1-5).png	← Ejecución de Código
└─ 5 Pylint P1-6 (desp ajuste cod).png	← Ejecución de Pylint
└─ 6 Ejec. TC1-4 (FINAL).png	← Ejecución de Código
└─ 7 Ejec. TC2.png	← Ejecución de Código
...	
└─ 7 Ejec. TC7.png	← Ejecución de Código

En el caso específico del caso de prueba TC1, el archivo de prueba contenía un dato incorrecto, con un carácter “s” que el profesor posteriormente eliminó, por lo que hay 2 screenshots para su ejecución: 2 Ejec. TC.png y 2 Ejec. TC1-2.png.

El archivo 4 Ejec. TC1-3 (desp. de pylint P1-5).png fue la última ejecución del caso de prueba TC1, luego de una corrección de un warning de Pylint.

Finalmente, los últimos ajustes confirmados por el profesor llevaron a ejecutar nuevamente pylint para asegurar que no hubiera errores (5 Pylint P1-6 Después de ajustes de código.png), y **la ejecución final del TC1 quedó registrada en el archivo 6 Ejec. TC1-4 (FINAL).png**

A partir de ahí, **los archivos de ejecución de los casos de prueba 2 al 7 quedaron en los archivos 7 Ejec. TC2.txt a 7 Ejec. TC7.txt**

Verificación con pylint

El código fue analizado, antes de ser ejecutado, con la herramienta pylint, obteniendo recomendaciones de tipo Refactor (R) principalmente y de tipo Warning (W), las cuales no afectan la funcionalidad del programa.

Dichas recomendaciones fueron implementadas y documentadas mediante capturas de pantalla. Los nombres de los archivos llevan un numeral al inicio seguido por la palabra Pylint. Están intercalados con los ejercicios de ejecución para indicar en qué orden se ejecutó cada cosa:

- pylint = se ejecutó pylint sobre el código
- Ejec = se ejecutó el código con un archivo de prueba

Programa 2 – Converter

Descripción del problema:

El ejercicio consiste en desarrollar un programa en Python que lea un archivo de texto con valores numéricos y convierta cada uno de ellos a base binaria y hexadecimal, manejando datos inválidos (excluyéndolos) y reportando los resultados tanto en pantalla como en archivos de salida.

Requerimientos recibidos

Los requerimientos implementados son los detallados en el documento de la asignación y que especifican:

Req1. The program shall be invoked from a command line. The program shall receive a file as parameter. The file will contain a list of items (presumable numbers).

Req 2. The program shall convert the numbers to binary and hexadecimal base. The results shall be print on a screen and on a file named ConversionResults.txt.

All computation **MUST** be calculated using the basic algorithms, not functions or libraries.

Req 3. The program shall include the mechanism to handle invalid data in the file. Errors should be displayed in the console and the execution must continue.

Req 4. The name of the program shall be
convertNumbers.py

Req 5. The minimum format to invoke the program shall be as follows:
python convertNumbers.py fileWithData.txt

Req 6. The program shall manage files having from hundreds of items to thousands of items.

Req 7. The program should include at the end of the execution the time elapsed for the execution and calculus of the data. This number shall be included in the results file and on the screen.

Req 8. Be compliant with PEP8.

Consideraciones y especificaciones adicionales sobre la implementación

Adicional a los requerimientos de la asignación, tomé decisiones sobre la implementación para poder cumplir con los requerimientos y con las salidas esperadas. A continuación los detallo.

1. Manejo de archivos de entrada

El archivo de entrada se procesa línea por línea.

Cada línea se conserva como un token de entrada, incluso si es inválida. Se considera como un token porque inicialmente no sabemos si es un número.

Sólo convertiremos los números enteros, dados los datos de prueba, y que no se especifica que se tendrán números reales.

2. Manejo y representación de datos inválidos

Si un valor no es entero, está vacío o no puede convertirse correctamente, se marca como inválido.

El número de líneas inválidas detectadas durante la lectura del archivo se muestra al final del archivo de resultados y también se imprime en pantalla.

Representación de valores inválidos

- En la salida, tanto en la columna BIN como en HEX, se coloca #VALUE!
- En la consola se muestra un mensaje de error indicando el número de línea y el valor inválido.

3. Formato del archivo de salida

El **archivo de resultados** contiene **cuatro columnas**, en el siguiente orden:

- **ITEM**: Numerador consecutivo que inicia en 1.
- **TCx**: Nombre del caso de prueba (TCx). Se lee del nombre del archivo de entrada.
- **BIN**: Conversión del valor a binario o #VALUE! si es inválido.
- **HEX**: Conversión del valor a hexadecimal o #VALUE! si es inválido.

4. Conversión de números negativos

Para los números negativos, el programa utiliza **representación en complemento a dos**, de acuerdo con los resultados esperados proporcionados:

- **Binario**: complemento a dos en **10 bits**.
- **Hexadecimal**: complemento a dos en **40 bits** (10 dígitos hexadecimales).

Esta decisión garantiza consistencia con las salidas de referencia y refleja el mecanismo estándar de representación de números negativos en sistemas computacionales.

5. Conversión de números reales

Aunque es posible convertir números reales a binario y hexadecimal mediante algoritmos adicionales o representaciones de punto flotante, este programa no lo contempla, ya que:

- el requerimiento se enfoca en conversiones básicas,
- los archivos de prueba contienen valores enteros,
- y el manejo de punto flotante no forma parte del alcance del ejercicio.

Entonces, los valores decimales se consideran datos inválidos y se reportan como #VALUE!.

6. Nombre del archivo de código fuente

El requerimiento solicita que el archivo de código fuente tenga el nombre **convertNumbers.py**, cuya estructura **no es adecuada de acuerdo** a los estándares de **PEP8**, por lo que **se cambió a** notación snake case: **convert_numbers.py**

Ejecución del programa

El programa se ejecutó desde la línea de comandos en la consola, como fue indicado, recibiendo como parámetro el archivo de prueba correspondiente (TC1.txt, TC2.txt... TC7.txt).

Ejemplo:

```
python convert_numbers.py ..\Test\TC1.txt
```

Resultados obtenidos

El nombre del archivo de salida es el solicitado en los requerimientos, **ConversionResults**, y para facilitar la trazabilidad de los casos de prueba, **el nombre del archivo de salida incluye el identificador del archivo de entrada**, generando un archivo de resultados independiente por cada caso de prueba.

Ejemplo:

Entrada: TC1.txt

Salida: **ConversionResultsTC1.txt**

En el **subfolder Screenshots del folder Results** se encuentran las **capturas de pantalla de los resultados de las pruebas, Siguiendo las reglas de numerales al inicio y nombres que incluyen Ejec** para indicar ejecución, nombres que incluyen Pylint para indicar pruebas.

Los archivos de resultados y screenshots se encuentra en los siguientes directorios, y el numeral al inicio indica orden de ejecución:

.	
.	
.	
P2	← Programa 1
└─ Source	← Código fuente
└─ Test	← Código fuente
└─ Results	← Resultados pbas: .txt y screenshots
└─ StatisticsResultsTC1.txt	← Resultados del TC1
...	
└─ StatisticsResultsTC4.txt	← Resultados del TC7
└─ Screenshots	← Capturas de pantalla
└─ 1 Pylint P2-1.png	← Ejecución de Pylint
...	
└─ 1 Pylint P2-3.png	← Ejecución de Pylint
└─ 2 Ejec. TC1.png	← Ejecución de Código
...	

Verificación con pylint

El código fue analizado, antes de ser ejecutado, con la herramienta pylint, obteniendo recomendaciones de tipo Convention (C, de estilo y formato) y Warning (W), las cuales no afectan la funcionalidad del programa.

Las recomendaciones fueron implementadas y documentadas mediante capturas de pantalla.

Programa 3 – Count Words

Descripción del problema:

En este caso se pide desarrollar un programa en Python que lea un archivo de texto con palabras, que pueden tener espacios entre ellas, e identifique las palabras distintas en el archivo así como el conteo de cada una, y reporte los resultados tanto en pantalla como en un archivo de salida.

Requerimientos recibidos

Los requerimientos implementados son los detallados en el documento de la asignación y que especifican:

Req1. The program shall be invoked from a command line. The program shall receive a file as parameter. The file will contain a words (presumable between spaces).

Req 2. The program shall identify all distinct words and the frequency of them (how many times the word “X” appears in the file). The results shall be print on a screen and on a file named WordCountResults.txt.

All computation **MUST** be calculated using the basic algorithms, not functions or libraries.

Req 3. The program shall include the mechanism to handle invalid data in the file. Errors should be displayed in the console and the execution must continue.

Req 4. The name of the program shall be
wordCount.py

Req 5. The minimum format to invoke the program shall be as follows:
python wordCount.py fileWithData.txt

Req 6. The program shall manage files having from hundreds of items to thousands of items.

Req 7. The program should include at the end of the execution the time elapsed for the execution and calculus of the data. This number shall be included in the results file and on the screen.

Req 8. Be compliant with PEP8.

Consideraciones y especificaciones adicionales sobre la implementación

Adicional a los requerimientos de la asignación, se recibieron precisiones, y se tomaron decisiones sobre la implementación, para poder cumplir con los requerimientos y con las salidas esperadas.

A continuación se detalla cuáles fueron.

1. Tokenización y limpieza de datos

Para el análisis del texto, definí el concepto de token como cada secuencia de caracteres separada por espacios o caracteres en blanco. Es decir, en una línea podría haber más de un token, es decir, de una palabra.

Posteriormente, cada token es normalizado mediante:

- conversión a minúsculas,
- eliminación de signos de puntuación comunes al inicio y al final de la palabra.

Los tokens que, después de este proceso de limpieza, resultan vacíos son considerados inválidos, se reportan en consola y no se incluyen en el conteo de frecuencias..

2. Palabras distintas vs. tokens válidos

En el reporte que genero presento dos métricas diferentes:

- **Tokens válidos:** número total de palabras procesadas correctamente, incluyendo una misma palabra repetidas todas las veces que aparezca en el texto.
- **Palabras distintas:** número de palabras únicas identificadas en el archivo, sin considerar repeticiones.

Esta distinción permite interpretar correctamente los resultados y facilita la comparación con **los archivos de resultados proporcionados**, en los cuales **el total de palabras procesadas suele reportarse como “Grand Total”**.

3. Ordenamiento de los resultados

El enunciado del ejercicio no especifica un criterio de ordenamiento para la presentación de los resultados.

Por esta razón, usé un ordenamiento alfabético ascendente por palabra, con el objetivo de:

- coincidir con el orden observado en los archivos de resultados proporcionados
- mejorar la legibilidad del reporte sin afectar la funcionalidad solicitada.

4. Formato del reporte

El reporte generado contiene:

- una **columna Label**, que representa la palabra identificada,
- una **columna Count**, que indica la frecuencia de aparición de dicha palabra.

Adicionalmente, al final del reporte se incluyen:

- el **número de palabras distintas**,
- el **número de tokens válidos**,
- el **número de tokens inválidos detectados**,
- el **tiempo total de ejecución** del programa.

5. Manejo de errores

El programa está diseñado para manejar datos inválidos sin interrumpir su ejecución. Los errores detectados durante la limpieza de tokens se reportan en consola, mientras que el procesamiento del resto de los datos continúa de manera normal.

Ejecución del programa

El programa se ejecutó desde la línea de comandos en la consola, como fue indicado, recibiendo como parámetro el archivo de prueba correspondiente (TC1.txt, TC2.txt... TC7.txt).

Ejemplo:

```
python word_count.py ..\Test\TC1.txt
```

Resultados obtenidos

El nombre del archivo de salida es el solicitado en los requerimientos, **WordCountResults**, y para facilitar la trazabilidad de los casos de prueba, **el nombre del archivo de salida incluye el identificador del archivo de entrada**, generando un archivo de resultados independiente por cada caso de prueba.

Ejemplo:

Entrada: TC1.txt

Salida: **WordCountResultsTC1.txt**

En el **subfolder Screenshots del folder Results** se encuentran las **capturas de pantalla de los resultados de las pruebas, con nombres que incluyen Ejec** para indicar ejecución, del tipo 2 Ejec..., 4 Ejec... m 7 Ejec.... **Los numerales al inicio del nombre de archivo indican orden en la ejecución de los diferentes casos de prueba y de Pylint.**

Los archivos de resultados y screenshots se encuentra en los siguientes directorios, y el numeral al inicio indica orden de ejecución:

.	
.	
.	
P3	← Programa 1
└ Source	← Código fuente
└ Test	← Código fuente
└ Results	← Resultados pbas: .txt y screenshots
└ WordCountResultsTC1.txt	← Resultados del TC1
...	
└ WordCountResultsTC5.txt	← Resultados del TC7
└ Screenshots	← Capturas de pantalla
└ 1 Pylint P3-1.png	← Ejecución de Pylint
...	
└ 1 Pylint P3-3.png	← Ejecución de Pylint
└ 2 Ejec. TC1.png	← Ejecución de Código
...	
└ 2 Ejec. TC5.png	← Ejecución de Código

Al **comparar los resultados obtenidos con los archivos de referencia** para los distintos casos de prueba (TC1–TC5), vi que:

- las palabras identificadas y sus frecuencias coinciden
- los totales de tokens válidos corresponden al valor reportado como Grand Total en los archivos esperados
- las diferencias encontradas corresponden únicamente a aspectos de presentación o nomenclatura, no a errores funcionales

Verificación con pylint

El código fue analizado, antes de ser ejecutado, con la herramienta pylint, obteniendo recomendaciones de tipo Convention (C, de estilo y formato), Error (E) y Warning (W).

Dichas recomendaciones fueron implementadas y documentadas mediante capturas de pantalla. Los nombres de los archivos llevan un numeral al inicio seguido por la palabra Pylint.

Conclusiones / Reflexiones

La ejecución de estos ejercicios me permitió aplicar tanto revisiones manuales que generalmente aplico a mi código, y también usar una herramienta de análisis de pylint para revisar el cumplimiento de un estándar de implementación o guía de estilo como es PEP8.

El uso de dicha herramienta me permitió eliminar defectos que yo no había encontrado, la mayoría de estilo o warnings, pero también de uso de variables, lo cual es muy bueno. Aunque no detecta errores de lógica, porque de entrada no es su foco, sí permite tener al final código legible y más fácilmente mantenible.

Me permitió recordar también la importancia del detalle de los requerimientos para el desarrollador, pues son prácticamente su única entrada de información para generar una solución, y se tiende a asumir muchos aspectos que parecen pequeños pero pueden ser trascendentes en el funcionamiento de una aplicación.

Me llevo de tarea buscar herramientas para revisar código, y sobre todo, ideas de herramientas que se pueden usar e incluso desarrollar para estándares internos de una organización o de un equipo de desarrollo.

Referencias (FORMATO APA)

Spolsky, J. (2005). *Making Wrong Code Look Wrong – Joel on Software*. <https://www.joelonsoftware.com/2005/05/11/making-wrong-code-look-wrong/>

Van Rossum, G., Warsaw, B. & Coghlan, Alyssa (2013). *PEP 8 – Style Guide for Python Cod.* Convención de codificación de Python - PEP8 <https://peps.python.org/pep-0008/>

Wieggers, K. E., & Beatty, J. (2013). *Software Requirements*: (3rd ed.). Microsoft Press.